AI Agents Portfolio

Technical work sample for software engineering and AI training roles
GitHub: https://github.com/yueLight12/ai-agents-portfolio

Summary: A compact Python AI agent platform with FastAPI endpoints, CLI demos, reusable agent classes, structured outputs, a pluggable LLM provider layer, deterministic mock mode, and pytest coverage.

Project Purpose

This project demonstrates practical AI application engineering rather than a one-off prompt script. It separates agent logic, prompt responsibilities, model-provider access, API transport, CLI execution, examples, and tests so the system can be reviewed, extended, and evaluated.

What The Reviewer Can Inspect

Agent Design

- `interview-coach`: tailored pitch, likely questions, STAR stories, gap risks.
- `research-brief`: executive summary, findings, risks, recommendations, next steps.
- `csv-insight`: data checks, hypotheses, feature ideas, charts, analysis plan.

Engineering Surface

- FastAPI service for HTTP execution.
- CLI for local JSON-based demos.
- Pydantic schemas for response contracts.
- Pytest tests for key behavior.

Technical Stack

- **Language:** Python 3.10+
- **API:** FastAPI
- **Validation:** Pydantic
- **LLM integration:** OpenAI SDK behind a provider abstraction
- **Testing:** Pytest
- **Packaging:** `pyproject.toml` with editable install support

Relevant Design Decisions

- **Provider abstraction:** business logic does not directly depend on one model vendor.
- **Mock mode:** the app can run and be tested without a live API key or network dependency.
- **Structured outputs:** agent responses are returned as dictionaries suitable for downstream evaluation or UI integration.
- **Shared core logic:** the same agent registry powers both the API and CLI.
- **Small review surface:** the repo is intentionally compact so reviewers can understand the architecture quickly.

Repository Structure

```
ai-agents-portfolio/
  src/ai_agents_portfolio/
    agents/
    app.py
    cli.py
    config.py
    llm.py
    registry.py
    schemas.py
  examples/
  tests/
  pyproject.toml
  README.md
  CODE_SAMPLE.md
```

How To Run Locally

```
cd ai-agents-portfolio
python -m venv .venv
.venv\Scripts\activate
pip install -e .[dev]
python -m pytest
```

Run the API:

```
uvicorn ai_agents_portfolio.app:app --reload --port 8010
```

Run the CLI:

```
python -m ai_agents_portfolio.cli list
python -m ai_agents_portfolio.cli run interview-coach --input examples/interview_input.json
```

Verification

The local test suite was run successfully with `python -m pytest`. Result: **4 tests passed**.

Why This Fits AI Training Work

The project shows prompt-driven workflow design, structured output expectations, reproducible local behavior, and clean separation between AI behavior and application plumbing. These are useful qualities for evaluating, improving, and operationalizing AI-assisted systems.

Prepared as a concise code sample. Full source code is available at the GitHub link above.